

# OTA System - Implementation Files

This PDF contains all the Python files for the OTA (Over-The-Air) system implementation.

## | ota\_project/server/ota\_server.py

```
class OTACloudServer:
    def __init__(self):
        self.firmware_store = {}
        self.manifests = {}
        self.campaigns = {}

    def authenticate_vehicle(self, vehicle_id: str) -> bool:
        return vehicle_id in ["vehicle_001", "vehicle_002"]

    def add_firmware(self, ecu_name: str, version: str, data: bytes, checksum: str):
        key = f"{ecu_name}_{version}"
        self.firmware_store[key] = data
        self.manifests[key] = {
            "version": version,
            "ecu": ecu_name,
            "size": len(data),
            "checksum": checksum,
            "url": f"https://ota-server/{ecu_name}/{version}/update.bin"
        }

    def handle_check_update(self, vehicle_id: str, current_version: str) -> dict:
        if not self.authenticate_vehicle(vehicle_id):
            return {"status": "unauthorized"}

        for key, manifest in self.manifests.items():
            if manifest["version"] > current_version:
                return {"update_available": True, "manifest": manifest}
        return {"update_available": False, "manifest": None}
```

## | ota\_project/client/ota\_client.py

```
import hashlib

class UpdateChecker:
    def __init__(self, server):
        self.server = server

    def check_for_update(self, vehicle_id: str, current_version: str) -> dict:
        response = self.server.handle_check_update(vehicle_id, current_version)
        return response

class DownloadManager:
    def __init__(self):
        self.storage = {}

    def download_package(self, url: str, manifest: dict) -> bool:
        try:
            self.storage[manifest["ecu"]] = {
                "manifest": manifest,
                "data": b"dummy_firmware_binary_data"
            }
            return True
        except Exception:
            return False

class VerificationModule:
    def verify_package(self, firmware_data: bytes, expected_checksum: str) -> bool:
        calculated_checksum = hashlib.md5(firmware_data).hexdigest()
        return calculated_checksum == expected_checksum
```

## | ota\_project/bootloader/bootloader.py

```
class Bootloader:
    def __init__(self, target_ecu):
        self.target_ecu = target_ecu

    def receive_firmware(self, firmware_data: bytes):
        if self.validate_signature(firmware_data):
            self.flash_firmware(firmware_data)
        else:
            self.rollback()

    def validate_signature(self, firmware_data: bytes) -> bool:
        return True

    def flash_firmware(self, firmware_data: bytes):
        self.target_ecu.update_firmware(firmware_data)

    def rollback(self):
        pass

class TargetECU:
    def __init__(self, name):
        self.name = name
        self.firmware = None

    def update_firmware(self, new_firmware: bytes):
        self.firmware = new_firmware
        print(f"[{self.name}] Firmware update successfully installed.")
```

## | ota\_project/test/test\_ota.py

```
from server.ota_server import OTACloudServer
from client.ota_client import UpdateChecker, DownloadManager, VerificationModule
from bootloader.bootloader import Bootloader, TargetECU

def test_ota_workflow():
    server = OTACloudServer()
    server.add_firmware("ADAS", "1.2.0", b"firmware_data", "5d41402abc4b2a76b9719d911017c592")

    target_ecu = TargetECU("ADAS")
    checker = UpdateChecker(server)
    download_manager = DownloadManager()
    verification_module = VerificationModule()
    bootloader = Bootloader(target_ecu)

    response = checker.check_for_update("vehicle_001", "1.0.0")
    assert response["update_available"] is True

    manifest = response["manifest"]
    download_manager.download_package(manifest["url"], manifest)

    firmware_data = b"firmware_data"
    is_valid = verification_module.verify_package(firmware_data, manifest["checksum"])
    assert is_valid is True

    bootloader.receive_firmware(firmware_data)
    assert target_ecu.firmware == firmware_data

if __name__ == "__main__":
    test_ota_workflow()
    print("OTA flow tested successfully!")
```